

Praktikumsaufgabe 5 – Reviews und Review der Anforderungen

Aufgabe 1: Checkliste für Anforderungsreviews

Die folgende Checkliste prüft die Qualität der Anforderungsspezifikation des KlausurGenerators. Die Qualitätsmaße beziehen sich auf die Kriterien nach C. Rupp (SMART-Kriterien für Anforderungen: Spezifisch, Messbar, Atomar, Realisierbar, Testbar sowie Eindeutigkeit, Vollständigkeit, Konsistenz).

Nr.	Prüfungspunkt	Qualitätsmaß (Rupp)	OK?	Befund / Notiz
1	Sind alle Anforderungen eindeutig und unmissverständlich formuliert?	Eindeutigkeit	J	Einige User Stories sind vage (z.B. "nach definierten Kriterien")
2	Sind alle Anforderungen testbar/verifizierbar?	Testbarkeit	N	FA: "Eine Klausur generieren" – kein Kriterium, wann erfolgreich
3	Sind alle Anforderungen vollständig (keine Lücken, keine TODO)?	Vollständigkeit	N	Kein Passwort-Reset, kein Session-Timeout definiert
4	Sind alle Anforderungen konsistent (keine Widersprüche)?	Konsistenz	N	Nicht-Ziel: kein Daten-Export – aber FA Low Priority: "Aufgabensammlung exportieren" (Widerspruch)
5	Ist jede Anforderung atomar (behandelt nur einen Aspekt)?	Atomarität	J	User Stories weitgehend atomar
6	Sind alle Anforderungen korrekt priorisiert (High/Medium/Low)?	–	J	Priorisierung vorhanden
7	Sind Abhängigkeiten zwischen Anforderungen dokumentiert?	–	N	Keine Abhängigkeitsmatrix vorhanden
8	Sind alle nicht-funktionalen Anforderungen messbar spezifiziert?	Messbarkeit	N	NFAs wie "intuitive Benutzeroberfläche" nicht messbar
9	Sind alle Akteure (Nutzer, System) klar benannt?	–	J	Akteur "Nutzer" durchgehend verwendet
10	Ist jede Anforderung eindeutig identifizierbar (ID)?	Eindeutigkeit	N	Keine eindeutigen Anforderungs-IDs vergeben
11	Sind alle Fachbegriffe im Dokument definiert oder referenziert?	–	N	Bloom'sche Taxonomie kurz erklärt, aber kein Glossar
12	Sind Randbedingungen und Einschränkungen explizit aufgeführt?	–	J	Nicht-Ziele und NFAs vorhanden
13	Sind alle Use Cases im Use-Case-Diagramm mit FAs konsistent?	Konsistenz	N	UC "Presets anzeigen" nicht in FA-Liste erwähnt
14	Ist das Datenschema vollständig und mit den FAs abgestimmt?	–	J	Datenbankschema vorhanden
15	Sind Fehlerfälle und Ausnahmen in den Anforderungen berücksichtigt?	Vollständigkeit	N	Fehlerfälle kaum beschrieben (nur Login-Fehlermeldung)

Aufgabe 2: Checkliste für Reviews von Anwendungsfällen

Die folgende Checkliste wird auf den Use-Case-Bereich der Anforderungsspezifikation angewendet (Use-Case-Diagramm, Abbildung 2).

Nr.	Prüfungspunkt	OK?	Befund / Notiz
1	Ist der Hauptpfad (Happy Path) vollständig definiert?	J	Hauptpfad bei meisten UCs erkennbar
2	Sind alle Alternativpfade und Ausnahmepfade beschrieben?	N	Keine Alternativpfade dokumentiert
3	Ist der Auslöser (Trigger) des Use Cases angegeben?	N	Trigger fehlen in der UC-Beschreibung
4	Sind Vor- und Nachbedingungen (Pre-/Postconditions) definiert?	N	Keine Pre-/Postconditions vorhanden
5	Ist der primäre Akteur eindeutig benannt?	J	"Nutzer" konsistent verwendet
6	Gibt es sekundäre Akteure (z.B. Datenbank) und sind diese beschrieben?	J	DB und Nutzerverwaltung im UC-Diagramm sichtbar
7	Sind alle <<include>>- und <<extend>>-Beziehungen korrekt verwendet?	N	<<include>> "Aufgabentyp wählen" – unklar ob include oder extend
8	Sind die Use Cases atomar (ein Ziel pro UC)?	J	UCs überwiegend atomar
9	Ist das Ziel des Actors aus dem UC-Namen erkennbar?	J	z.B. "Klausur erstellen", "Aufgabe löschen"
10	Sind alle UCs im Diagramm in der FA-Liste wiederzufinden?	N	UC "Presets anzeigen" fehlt in FA-Liste
11	Sind die Systemgrenzen (System Boundary) klar definiert?	J	"Klausursoftware" als Systemgrenze im Diagramm
12	Stimmt die Granularität der UCs (nicht zu fein, nicht zu grob)?	J	Granularität angemessen

Aufgabe 3: Geeignete Reviewart

Empfehlung: Walkthrough

Für die vorliegende Anforderungsspezifikation wird ein Walkthrough als geeignete Reviewart vorgeschlagen. Die Begründung stützt sich auf folgende Überlegungen:

1. Studentisches Semesterprojekt: Das Dokument wurde von einem fünfköpfigen Studierendenteam im Rahmen eines Semesterprojekts (SoSe 2025) erstellt. Formale Review-Verfahren wie Inspektionen erfordern geschulte Reviewer und einen hohen organisatorischen Aufwand, der in einem Hochschulkontext unverhältnismäßig wäre.
2. Ziel: Wissenstransfer und Fehleraufdeckung: Ein Walkthrough ermöglicht es dem Autorenteam, das Dokument selbst vorzustellen und dabei Fragen und Kommentare der Teilnehmer aufzunehmen. Dies fördert das gemeinsame Verständnis der Anforderungen.
3. Umfang und Reife des Dokuments: Die Spezifikation ist strukturell vollständig (Einleitung, Architektur, FA/NFA, Datenschema, UI), weist aber inhaltliche Lücken und Widersprüche auf (vgl. Aufgabe 4), die im Walkthrough effizient diskutiert werden können.
4. Abgrenzung zu anderen Reviewarten:
 - Ein informelles Review (Peer Review) wäre zu unsystematisch für ein Hochschulprojekt, das bewertet wird.
 - Eine Inspektion (nach Fagan) wäre zu aufwändig und erfordert ausgebildete Moderatoren sowie mehrstufige Prozesse.

- Ein technisches Review wäre geeignet, wenn der Fokus auf technischen Entscheidungen (z.B. Architekturvalidierung) liegen würde – hier steht aber die Anforderungsqualität im Vordergrund.

Fazit: Ein Walkthrough mit dem Auftraggeber (Dozent) und den Entwicklern bietet das beste Kosten-Nutzen-Verhältnis. Der Autor führt durch das Dokument, Reviewer stellen Fragen und identifizieren Probleme, die protokolliert und nachverfolgt werden.

Aufgabe 4: Review der Anforderungsspezifikation (Notizen)

Basierend auf der Checkliste aus Aufgabe 1 wurden folgende Auffälligkeiten, Mängel und Fehler in der Anforderungsspezifikation identifiziert:

Fehler / Widersprüche (schwerwiegend)

F1 – Widerspruch Export-Funktion: In Abschnitt 1.4 (Nicht-Ziele) steht explizit: "die Software bietet keine Möglichkeit zum Data Sharing, d.h. die Aufgaben in einer Datenbank sind lokal gespeichert und können nicht exportiert und importiert werden." Gleichzeitig ist als Low-Priority-FA definiert: "Als Nutzer kann ich die Aufgabensammlung exportieren, um sie auf einem anderen Computer wieder zu importieren." – Dies ist ein direkter Widerspruch.

F2 – Use Case ohne FA-Entsprechung: Im Use-Case-Diagramm (Abb. 2) ist der Use Case "Presets anzeigen" eingezeichnet, der in keiner funktionalen Anforderung erwähnt wird. Unklar, was darunter zu verstehen ist.

Mängel (mittelschwer)

M1 – Fehlende Anforderungs-IDs: Keine einzige funktionale oder nicht-funktionale Anforderung besitzt eine eindeutige ID (z.B. FA-001). Dadurch ist eine Rückverfolgbarkeit (Traceability) nicht möglich.

M2 – NFAs nicht messbar: Die NFA "intuitive Benutzeroberfläche" (Abschnitt 2.3) ist nicht verifizierbar. Es fehlen konkrete Usability-Kriterien (z.B. Nutzertests, maximale Klicktiefe).

M3 – Fehlende Fehlerfälle: Außer der Login-Fehlermeldung werden keine Fehlerfälle oder Ausnahmebehandlungen spezifiziert. Was passiert z.B., wenn die Datenbank keine passenden Aufgaben für eine Klausur enthält?

M4 – Keine Abhängigkeiten zwischen Anforderungen: Es fehlt eine Beschreibung, welche FAs voneinander abhängen (z.B. "Klausur erstellen" setzt "Aufgabe hinzufügen" voraus).

M5 – Geschätzter Zeitrahmen fehlt: Das Attribut "Geschätzte Zeit für die Lösungsfindung" aus dem Anforderungsdokument (Anhang A) taucht im Datenbankschema als "Zeit" auf, wird aber nirgends funktional beschrieben (wie wird es genutzt? In welcher Einheit?).

Auffälligkeiten (leicht)

A1 – Tippfehler in Abbildung 10: Der Screen "Klausur erstellen" enthält "Mutiple-Choice" statt "Multiple-Choice".

A2 – Titelseite enthält schwarzen Block: Auf der Titelseite (Anhang B) ist ein schwarzer Bereich sichtbar, vermutlich ein fehlgeschlagenes Bild.

A3 – "Klassurgenerator" vs. "Klausurgenerator": In Abbildung 7 steht "Klasurgenerator" (ein 's' zu wenig).

A4 – Nicht alle Diagramme beschriftet: Das Zustandsdiagramm (Abb. 4) ist sehr kleinteilig und kaum lesbar. Eine höhere Auflösung oder Beschreibung der Zustände wäre hilfreich.

Aufgabe 5: Review eines Anwendungsfalls (Notizen)

Gewählter Use Case: "Aufgabe erstellen" (aus Use-Case-Diagramm, Abb. 2). Dieser UC wurde gewählt, da er zentral für das System ist und im Dokument am detailliertesten durch UI-Mockups beschrieben wird.

Fehler / Widersprüche

F1 – <<include>>-Beziehung unklar: "Aufgabe erstellen" hat eine <<include>>-Beziehung zu "Aufgabentyp wählen". Gemäß UML-Notation bedeutet <<include>>, dass der eingeschlossene UC immer ausgeführt wird. Es stellt sich jedoch die Frage: Gilt "Lösung erstellen" als obligatorischer Schritt (<<include>>) oder nur als optionaler Erweiterungsfall (<<extend>>)? Aus den UI-Mockups geht hervor, dass der zweite Screen vom Aufgabentyp abhängt – dies deutet eher auf <<extend>> hin.

Mängel

M1 – Kein textueller UC-Template: Es gibt kein schriftliches UC-Template (Name, Akteur, Trigger, Vorbedingung, Hauptpfad, Alternativpfad, Nachbedingung). Das UC-Diagramm allein reicht für ein vollständiges Review nicht aus.

M2 – Fehlende Vorbedingung: Es ist nicht spezifiziert, ob der Nutzer eingeloggt sein muss, um eine Aufgabe zu erstellen (obwohl dies logisch vorausgesetzt wird).

M3 – Fehlende Nachbedingung: Es ist nicht beschrieben, in welchem Zustand sich das System nach erfolgreichem Abschluss befindet (z.B. "Aufgabe in Datenbank gespeichert, Nutzer sieht Bestätigungsmeldung").

M4 – Fehlende Alternativpfade: Was passiert, wenn der Nutzer beim Erstellen auf "Abbrechen" klickt? Was passiert bei fehlerhaften Eingaben (z.B. leerer Titel)?

M5 – Pflichtfelder nicht markiert: In den UI-Mockups (Abb. 9) ist nicht ersichtlich, welche Felder Pflichtfelder sind.

Auffälligkeiten

A1 – Modul als Freitexteingabe: Im Mockup "Aufgabe hinzufügen" (Abb. 9) ist "Modul" als freies Textfeld dargestellt. Dies steht im Widerspruch zum Klassendiagramm, wo Module als separate Entität modelliert sind. Besser wäre ein Dropdown.

A2 – "Anzahl der Punkte" fehlt in FA: Das UI-Feld "Anzahl der Punkte" erscheint im Mockup, taucht aber nicht als explizites Attribut in den Anforderungen (Anhang A) auf.